

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: <u>Ciencias de la Computación</u>
	Carrera: <u>Ingeniería de Software</u> Asignatura: <u>Análisis y Diseño de Software</u> NRC: <u>30723</u>
	Apellidos y nombres del estudiante: <u>Egas Jácome Steven David</u> <u>Obando Zapata Alejandro Xavier</u> <u>Qumbulco Carrion Juan Diego</u>

ANTECEDENTES

TransControl es un sistema web para gestionar viajes de transporte de carga pesada en Ecuador, desarrollado para reemplazar el manejo manual mediante WhatsApp, llamadas y hojas de cálculo. La SRS (Rev. 1.0, Febrero 2026) define tres módulos principales: Gestión de Viajes, Gestión de Transportistas y Monitoreo de Viajes.

Durante el análisis del proyecto se identificó la necesidad de aplicar patrones estructurales que permitan al sistema escalar sin romper el código existente. El módulo de Gestión de Transportistas maneja operaciones CRUD sobre entidades con distintas posibles implementaciones de persistencia. Además, la Documentación Habilitante (licencia tipo E, SOAT, matrícula, etc.) se repite para cada uno de los transportistas registrados, generando datos redundantes en memoria.

Ante estas necesidades se propone aplicar: el patrón Bridge para separar la lógica de servicio de la persistencia, y el patrón Flyweight para compartir los tipos de documentos habilitantes entre todos los transportistas, optimizando el uso de memoria.

OBJETIVO

Diseñar y proponer la aplicación de los patrones de diseño Bridge y Flyweight en el sistema TransControl, identificando las clases involucradas, el mapeo estructural y el código UML en PlantUML correspondiente a cada patrón.

DESARROLLO

1. Patrón Bridge

1.1 ¿Qué es el patrón Bridge?

Es un patrón de diseño estructural cuyo objetivo es separar la abstracción de su implementación. Al dividir una clase compleja en dos jerarquías independientes, permite que tanto la lógica de negocio como los detalles de implementación evolucionen de forma autónoma, sin afectarse entre sí.

1.2 Características

- Desacopla la abstracción de su implementación.
- Permite cambiar la implementación en tiempo de ejecución sin modificar la abstracción.
- Facilita agregar nuevas abstracciones e implementaciones de forma independiente.
- Aplica el principio de inversión de dependencias (DIP).

1.3 ¿Por qué usar Bridge en TransControl?

El patrón Bridge separa un sistema en dos jerarquías independientes: la abstracción, que define la interfaz y las operaciones visibles, y la implementación, que se encarga de la lógica interna. En TransControl, el módulo de Gestión de Transportistas necesita operar con distintas fuentes de datos (en memoria para desarrollo, base de datos para producción). Sin Bridge, cambiar el repositorio implicaría modificar toda la lógica de servicio. Con Bridge, el servicio no sabe cómo se persisten los datos —solo llama al repositorio a través de la interfaz.

1.4 Mapeo del patrón Bridge en TransControl

Elemento del patrón Bridge	Clase en el proyecto	Responsabilidad
----------------------------	----------------------	-----------------

<i>Abstraction</i>	<i>ViajeServicio</i>	<i>Define las operaciones generales del CRUD de viajes que usará la capa de presentación.</i>
<i>Refined Abstraction</i>	<i>ViajeServicioBasico</i>	<i>Implementa reglas básicas de validación (datos obligatorios, unicidad de ID) y delega el almacenamiento al repositorio.</i>
<i>Implementor</i>	<i>ViajeRepositorio</i>	<i>Interfaz que declara las operaciones concretas de persistencia: guardar, buscar, actualizar, cancelar, listarTodos.</i>
<i>Concrete Implementor</i>	<i>ViajeRepositorioMemoria</i>	<i>Guarda, busca, actualiza y cancela viajes en una lista en memoria (ArrayList<Viaje>).</i>
<i>Client</i>	<i>PantallaGestionViajes / Main</i>	<i>Conecta el servicio con la implementación y permite usar el menú por consola o interfaz Swing.</i>

1.5 Diagrama PlantUML del patrón Bridge aplicado

```

@startuml
interface ViajeRepositorio {
    + guardar(viaje: Viaje): boolean
    + buscarPorId(id: String): Optional<Viaje>
    + actualizar(id: String, datos): boolean
    + cancelar(id: String): boolean
    + listarTodos(): List<Viaje>
}

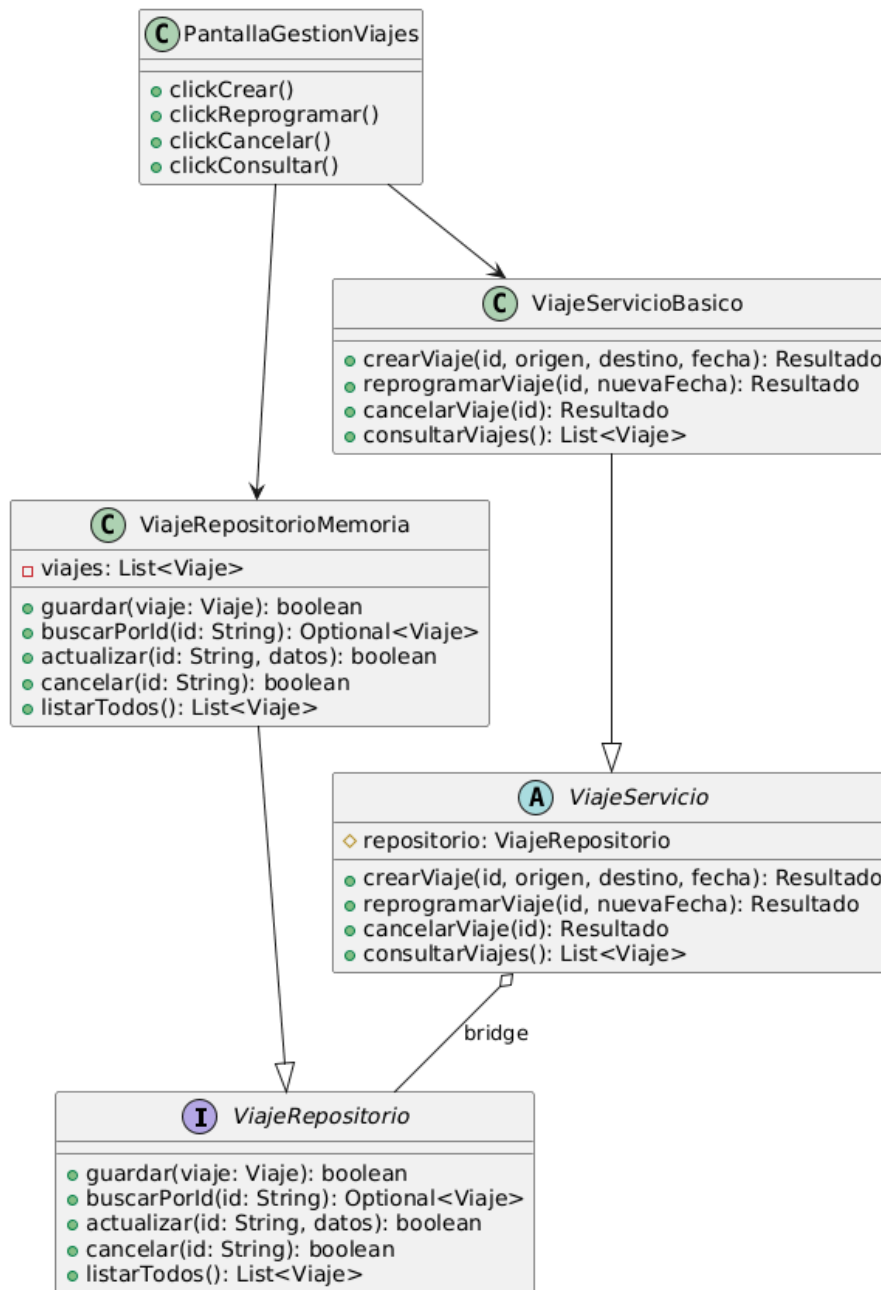
class ViajeRepositorioMemoria {
    - viajes: List<Viaje>
    + guardar(viaje: Viaje): boolean
    + buscarPorId(id: String): Optional<Viaje>
    + actualizar(id: String, datos): boolean
    + cancelar(id: String): boolean
    + listarTodos(): List<Viaje>
}
ViajeRepositorioMemoria --|> ViajeRepositorio

abstract class ViajeServicio {
    # repositorio: ViajeRepositorio
    + crearViaje(id, origen, destino, fecha): Resultado
    + reprogramarViaje(id, nuevaFecha): Resultado
    + cancelarViaje(id): Resultado
    + consultarViajes(): List<Viaje>
}

class ViajeServicioBasico {
    + crearViaje(id, origen, destino, fecha): Resultado
    + reprogramarViaje(id, nuevaFecha): Resultado
    + cancelarViaje(id): Resultado
    + consultarViajes(): List<Viaje>
}
ViajeServicioBasico --|> ViajeServicio
ViajeServicio o-- ViajeRepositorio : bridge

class PantallaGestionViajes {
    + clickCrear()
    + clickReprogramar()
    + clickCancelar()
    + clickConsultar()
}
PantallaGestionViajes --> ViajeServicioBasico
PantallaGestionViajes --> ViajeRepositorioMemoria
@enduml

```



2. Patrón Flyweight

2.1 ¿Qué es el patrón Flyweight?

El patrón Flyweight busca ahorrar memoria compartiendo objetos que tienen información repetida. En vez de crear muchos objetos iguales, reutiliza uno existente. Es un patrón de diseño estructural que permite mantener más objetos dentro de la cantidad disponible de RAM compartiendo las partes comunes del estado entre varios objetos, en lugar de mantener toda la información en cada objeto.

2.2 Características

- Reduce el consumo de memoria.
- Comparte datos comunes entre objetos (estado intrínseco).
- El estado único de cada instancia se pasa como parámetro (estado extrínseco).
- Mejora el rendimiento cuando existen miles de objetos similares.

2.3 ¿Por qué usar Flyweight en TransControl?

En TransControl, la Documentación Habilitante (sección 1.4.1 SRS) incluye tipos de documentos como licencia tipo E, SOAT, matrícula vehicular, revisión técnica, entre otros. Estos tipos comparten propiedades comunes (nombre del tipo, descripción legal, formato requerido) que se repiten para cada transportista. Si el sistema registra 500 transportistas con 6 documentos cada uno, sin Flyweight se crearían 3.000 objetos con datos redundantes. Con Flyweight, cada tipo de documento se instancia una sola vez y se reutiliza en todos los transportistas.

2.4 Mapeo del patrón Flyweight en TransControl

Elemento del patrón Flyweight	Clase en el proyecto	Responsabilidad
Flyweight Interface	DocumentoHabilitante	Define la operación <code>mostrarInfo(numeroDoc, fechaVenc, idTransportista)</code> que acepta el estado extrínseco como parámetro.
Concrete Flyweight	TipoDocumentoHabilitante	Almacena el estado intrínseco compartido: <code>tipoDoc</code> y <code>descripcionLegal</code> . Sus atributos son final (inmutables).
Flyweight Factory	FabricaDocumentos	Administra y reutiliza instancias de <code>TipoDocumentoHabilitante</code> mediante un <code>Map<String, DocumentoHabilitante></code> , evitando duplicados.
Client	GestorDocumentacionTransportista	Solicita documentos a la fábrica y pasa el estado extrínseco (número, fecha, ID del transportista) al llamar <code>mostrarInfo()</code> .

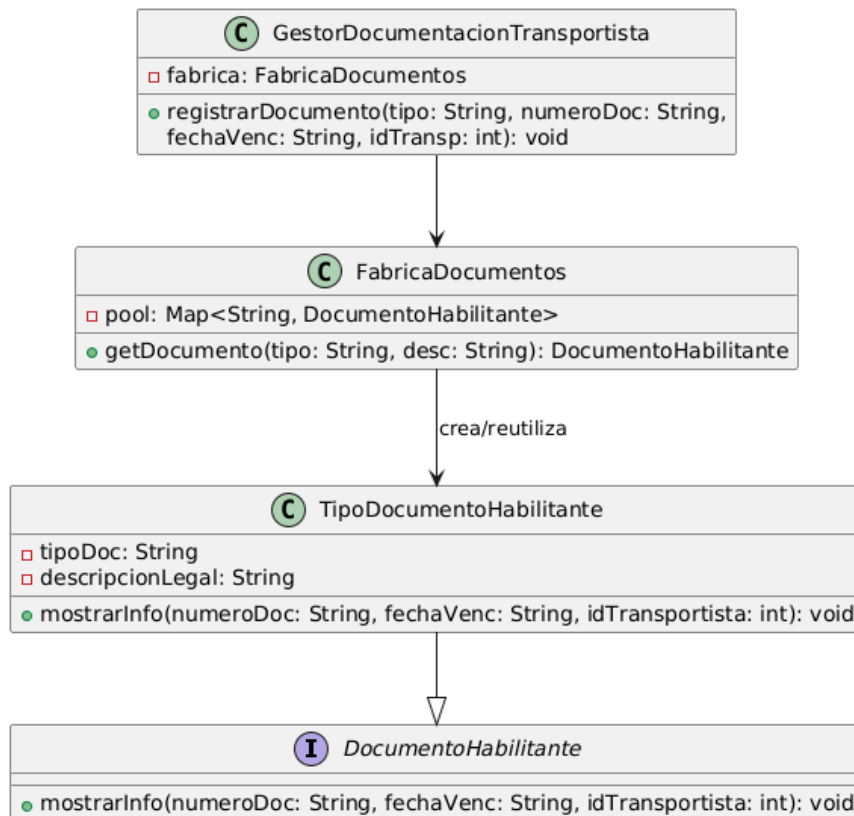
2.5 Diagrama PlantUML del patrón Flyweight aplicado

```
@startuml
interface DocumentoHabilitante {
    + mostrarInfo(numeroDoc: String, fechaVenc: String, idTransportista: int): void
}

class TipoDocumentoHabilitante {
    - tipoDoc: String
    - descripcionLegal: String
    + mostrarInfo(numeroDoc: String, fechaVenc: String, idTransportista: int): void
}
TipoDocumentoHabilitante --|> DocumentoHabilitante

class FabricaDocumentos {
    - pool: Map<String, DocumentoHabilitante>
    + getDocumento(tipo: String, desc: String): DocumentoHabilitante
}
FabricaDocumentos --> TipoDocumentoHabilitante : crea/reutiliza

class GestorDocumentacionTransportista {
    - fabrica: FabricaDocumentos
    + registrarDocumento(tipo: String, numeroDoc: String, fechaVenc: String, idTransp: int): void
}
GestorDocumentacionTransportista --> FabricaDocumentos
@enduml
```



CONCLUSIONES

El patrón Bridge permite a TransControl separar la lógica de validación del servicio de viajes de la lógica de persistencia, de modo que en versiones futuras se pueda cambiar el repositorio en memoria por uno con base de datos real sin modificar la capa de negocio, tal como prevé la sección 2.6 de la SRS.

El patrón Flyweight reduce el consumo de memoria al compartir los tipos de Documentación Habilitante entre todos los transportistas registrados, evitando crear objetos duplicados con los mismos datos legales para cada uno de los 6 documentos por transportista.

Ambos patrones mejoran la mantenibilidad y escalabilidad del sistema: Bridge mediante el desacoplamiento entre abstracción e implementación, y Flyweight mediante la reutilización de objetos compartidos en un pool administrado por la fábrica.

RECOMENDACIONES

Al implementar Bridge, definir ViajeRepositorio como una interfaz Java pura, sin lógica de negocio, para facilitar la creación de un ConcreteImplementor con JPA/Hibernate cuando TransControl migre a base de datos real.

Para Flyweight, usar ConcurrentHashMap en FabricaDocumentos si el sistema tendrá acceso concurrente multi-usuario, garantizando la integridad del pool de objetos compartidos en entornos con múltiples coordinadores activos simultáneamente.

Documentar claramente en el código qué atributos son intrínsecos (final, solo en TipoDocumentoHabilitante) y cuáles son extrínsecos (parámetros de mostrarInfo) para evitar que futuros desarrolladores rompan la inmutabilidad del Flyweight.

Sangolquí, a 01 de junio de 2026

